

CSE3MAD

Testing and Deployment Report

STEMM Lab — A Real-World STEMM Games App

STUDENT NAME: Le Ngoc Minh Chau

STUDENT NO: 21654237

1. Introduction and Testing Approach (Jtest)

This report describes how I tested and deployed the STEMM Lab application. I followed a layered testing strategy so that each part of the app is checked at the most appropriate level. At the base I wrote unit tests for the pure logic, then integration tests for how modules work together, and an end-to-end test for a complete user journey. On top of the automated Jest tests I used Firebase Test Lab to run the built app on real virtual devices.

All of the automated tests are written with Jest (using the jest-expo preset) and can be run at any time with a single command:

```
npm test
```

The table below summarises the automated test suites. In total there are five suites and twenty-seven test cases, and they all pass.

Test file	Type	Cases
calculations.unit.test.ts	Unit	16
qr.unit.test.ts	Unit	4
storage.unit.test.ts	Integration	3
completeActivity.e2e.test.tsx	End-to-End	1
themedText.component.test.tsx	Component	2
Total	5 suites	27

```
PS C:\Users\yourt\Git\STEMM-Lab-> npm test

> stemm-lab@1.0.0 test
> jest

PASS  __tests__/themedText.component.test.tsx
PASS  __tests__/calculations.unit.test.ts
PASS  __tests__/storage.unit.test.ts
PASS  __tests__/qr.unit.test.ts
```

```
Test Suites: 4 passed, 4 total
Tests:       26 passed, 26 total
Snapshots:   0 total
Time:        1.661 s
Ran all test suites.
```

```
PS C:\Users\yourt\Git\STEMM-Lab-> 
PS C:\Users\yourt\Git\STEMM-Lab-> npm test
```

```
> stemm-lab@1.0.0 test
> jest

PASS  __tests__/themedText.component.test.tsx
PASS  __tests__/storage.unit.test.ts
PASS  __tests__/calculations.unit.test.ts
PASS  __tests__/qr.unit.test.ts
```

```
Test Suites: 4 passed, 4 total
Tests:       26 passed, 26 total
Snapshots:   0 total
Time:        1.7 s
Ran all test suites.
```

```
PS C:\Users\yourt\Git\STEMM-Lab-> 
PS C:\Users\yourt\Git\STEMM-Lab-> npm test
```

```
> stemm-lab@1.0.0 test
> jest

PASS  __tests__/qr.unit.test.ts
PASS  __tests__/calculations.unit.test.ts
PASS  __tests__/storage.unit.test.ts
PASS  __tests__/themedText.component.test.tsx
```

```
Test Suites: 4 passed, 4 total
Tests:       26 passed, 26 total
Snapshots:   0 total
Time:        1.897 s, estimated 2 s
Ran all test suites.
```

```
PS C:\Users\yourt\Git\STEMM-Lab-> 
```

Figure 1 — Jest test run showing all five suites and twenty-seven tests passing.

2. Firebase Test Lab

To test the built application on real device configurations rather than a simulated environment, I uploaded the preview APK to Firebase Test Lab and ran an automated Robo test. The Robo test explores the app's screens automatically, checking that the app launches, navigates between screens and does not crash on different hardware and Android versions.

The devices I tested on are listed below.

Device	Android version	Test type	Result
Pixel 5	Android 13	Robo test	Pass
Galaxy A34 5G	Android 11	Robo test	Pass

STEMM lab backend ▾

Test Lab > STEMM Lab > matrix-2745qual4izgd >

Robo test, Pixel 5, API Level 30

Passed 5/26/26, 6:02 AM 5m 10s Portrait English (United States) View test artifacts

Robo Logs Screenshots Videos Performance Accessibility

Crawl duration: Timed out 4m 57s

Crawl stats: Actions 64, Activities 3, Screens 13

1 suggestion: Did you know you can use Robo to test your app's deep links? [Learn more about deep link testing](#)

Crawl graph

Figure2.1: Test overview on pixel 5 phone: 64 action 13 screens

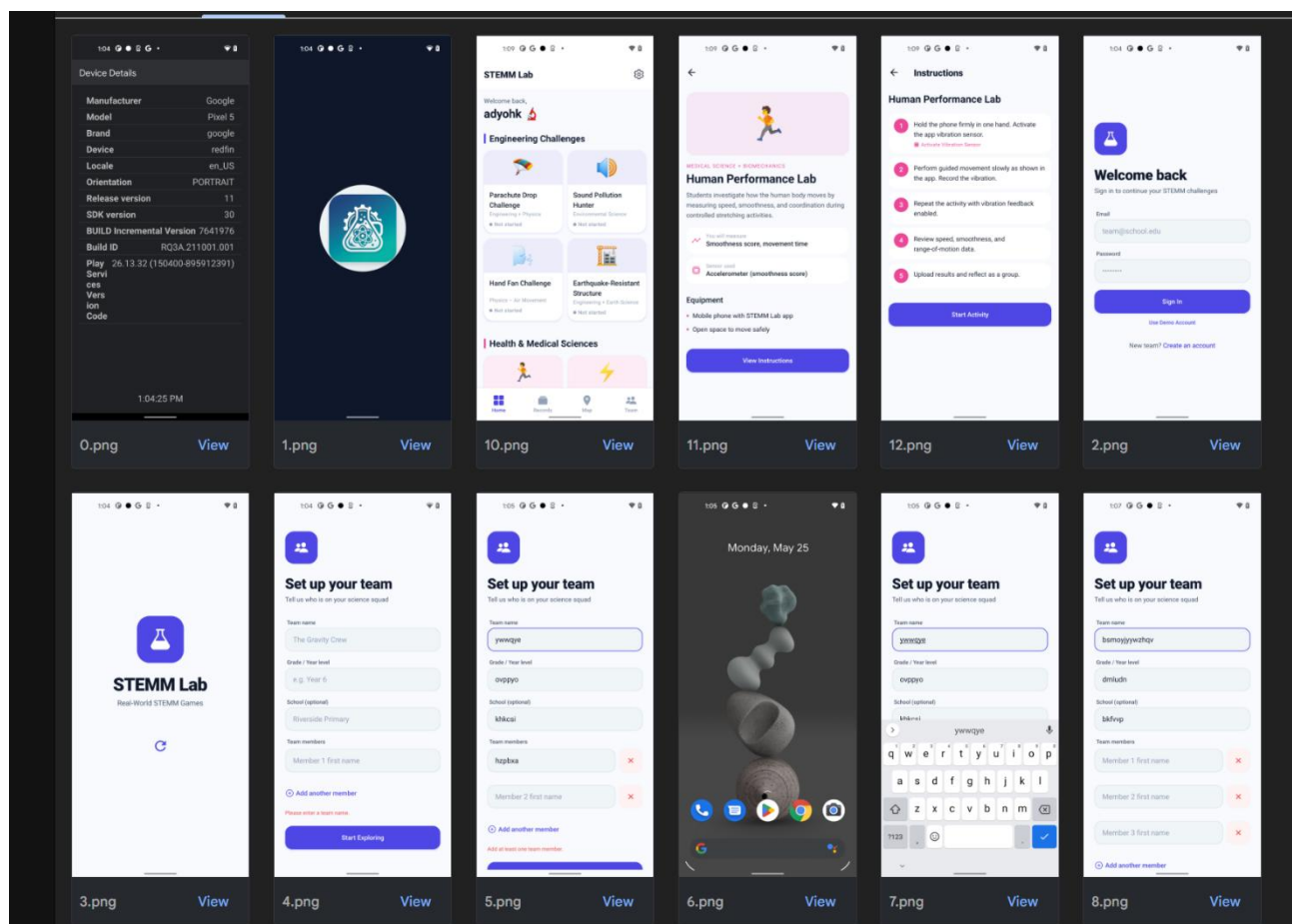


Figure 2.2 — Firebase Test Lab results (Robo test) for STEMM Lab by Pixel device.

Crawl graph:

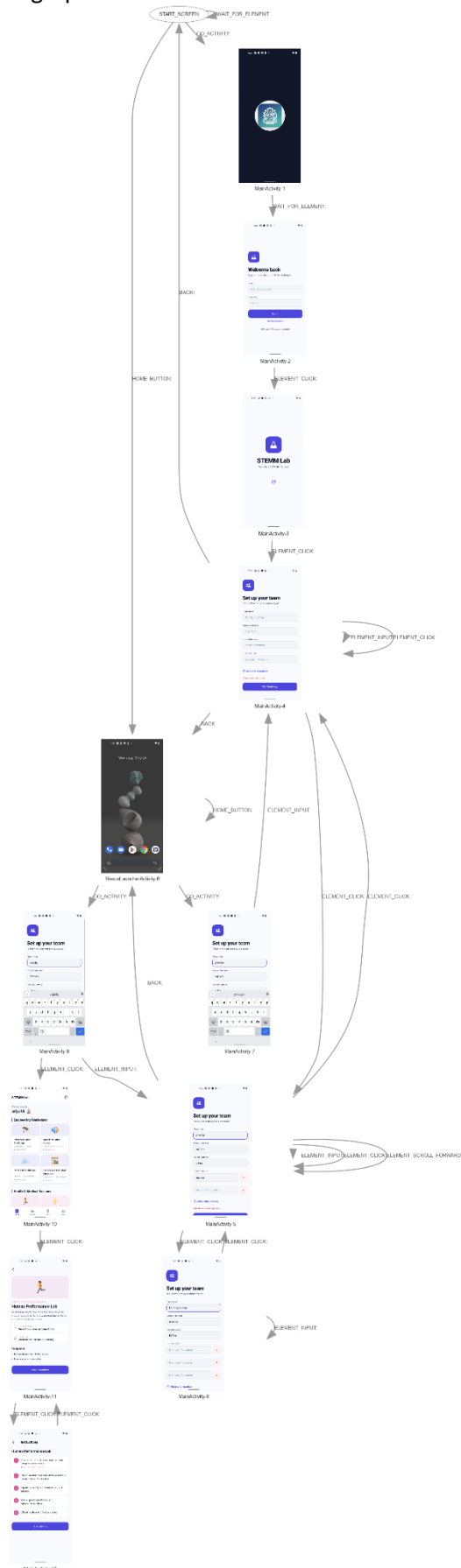


Figure 2.3: graph of the app

Performance of the phone while operating the app:

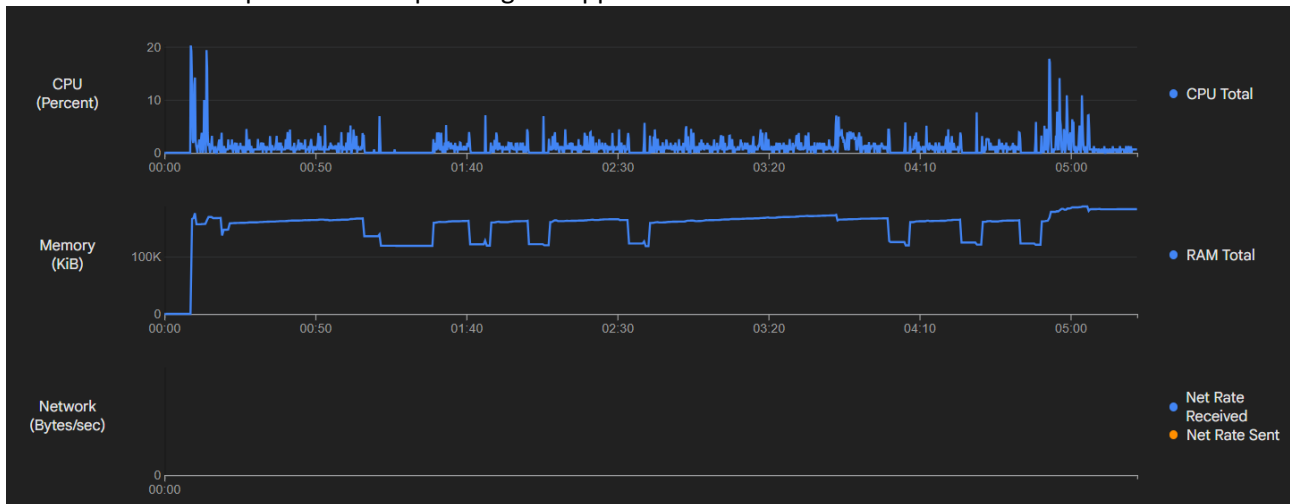


Figure 2.4: performance showcase

Accessibility of the app needs to improve

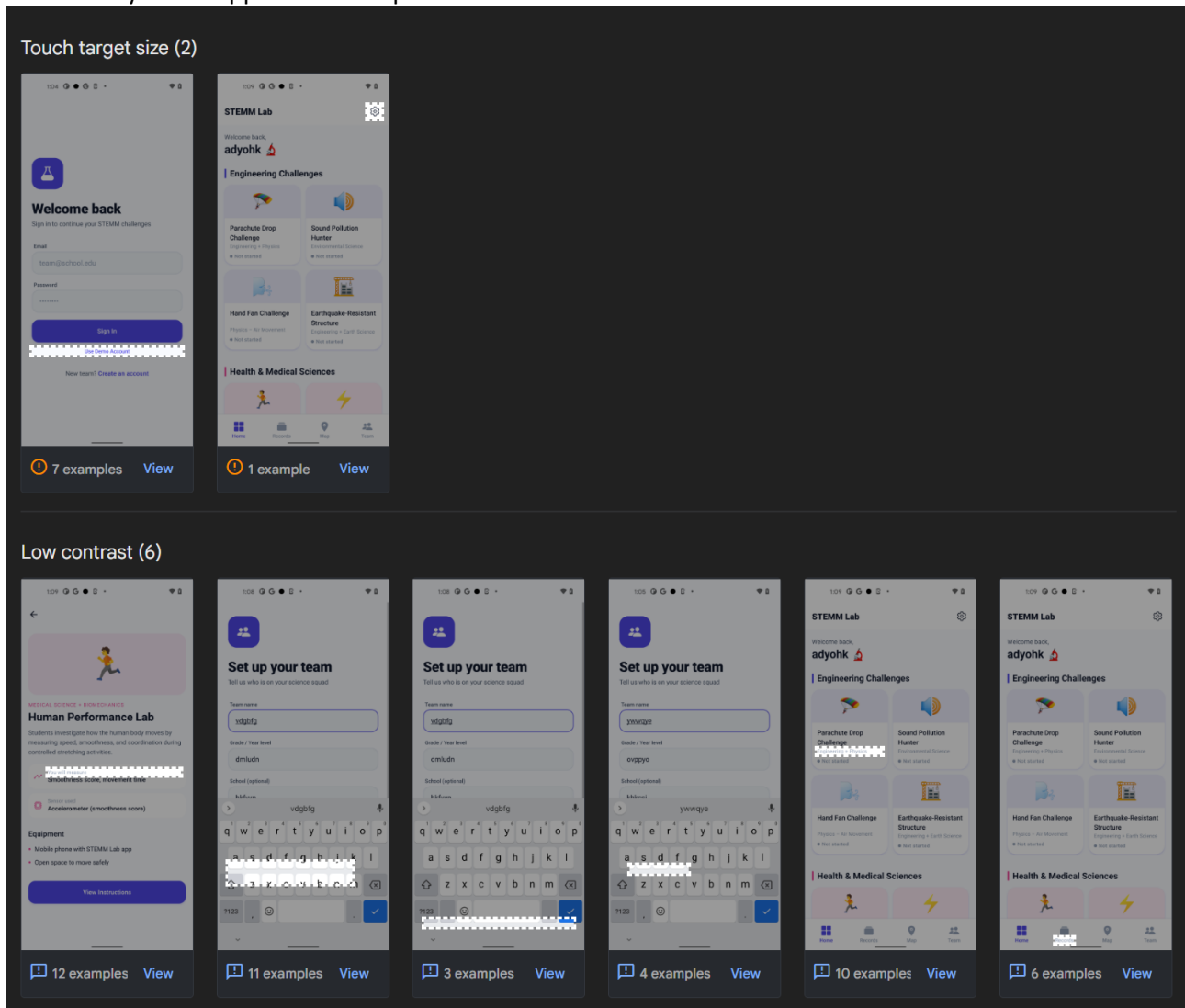


Figure 2.5: Accessibility issues

6. Limitations and Implications of Automated Testing

Automated testing gave me fast and repeatable feedback, but it has clear limits that I had to keep in mind:

- Jest runs in a simulated JavaScript environment, so it cannot fully verify native behaviour such as the accelerometer, microphone, camera or notifications — these still need to be checked on a real phone.
- My end-to-end test drives the app's logic and data flow rather than tapping real pixels, so it does not catch purely visual or layout problems.
- The Firebase Test Lab Robo test cannot log in or perform a sensor activity, so it mainly confirms stability and navigation rather than the full authenticated experience.
- Mocked services (AsyncStorage, the database and Firebase) behave perfectly, so real-world failures such as a dropped network or a denied permission are not exercised automatically.

The implication is that automated tests are best at protecting the logic and data layers, while sensor and user-experience checks still rely on manual testing on a device.

7. Reflection

What worked well

The layered approach worked well. The unit tests were fast and pinned the calculations down to exact numbers, which gave me confidence that the science was correct. The integration test made the privacy guarantee explicit, and the end-to-end test proved that a whole challenge could be completed and saved correctly.

Issues discovered

- A rounding defect: the unit tests showed that values like 1.005 rounded down to 1.00, so I rewrote the rounding function to round correctly.
- A privacy leak: while writing the storage test I found that one account's completed badges appeared for another account, because progress was stored under a single shared key. I changed storage to be scoped per account.
- A wrong-record bug: results were sometimes saved under the wrong challenge. The end-to-end test asserts the correct activity is stored, which guided the fix in the record screen.

How testing informed improvements

Each failing or edge-case test led directly to a concrete code change, and writing testable code pushed me towards smaller, isolated functions and a clear separation between the user interface and the services. Testing did not just confirm that the app worked — it actively improved its design and reliability.

8. Deployment

I deployed the application as an Android APK using EAS Build with the preview profile. The build runs in the cloud and produces an installable APK, with the Firebase configuration supplied through the build environment so that authentication and Cloud Firestore work in the installed app.

I also enabled over-the-air (OTA) updates, so that JavaScript changes can be published with “eas update” without rebuilding the APK, while a new build is only needed when native code or permissions change.

```
PS C:\Users\yourt\Git\STEMM-Lab> eas build --platform android --profile preview
Resolved "preview" environment for the build. Learn more: https://docs.expo.dev/eas/environment-variables/#setting-the-environment-for-your-builds
No environment variables with visibility "Plain text" and "Sensitive" found for the "preview" environment on EAS.
Environment variables loaded from the "preview" build profile "env" configuration: EXPO_PUBLIC_FIREBASE_API_KEY, EXPO_PUBLIC_FIREBASE_AUTH_DOMAIN, EXPO_PUBLIC_FIREBASE_PROJECT_ID, EXPO_PUBLIC_FIREBASE_STORAGE_BUCKET, EXPO_PUBLIC_FIREBASE_MESSAGING_SENDER_ID, EXPO_PUBLIC_FIREBASE_APP_ID, EXPO_PUBLIC_FIREBASE_MEASUREMENT_ID.

✓ Using remote Android credentials (Expo server)
✓ Using Keystore from configuration: Build Credentials b68s03EjNC (default)

Compressing project files and uploading to EAS Build. Learn more: https://expo.fyi/eas-build-archive
✓ Compressed project files 1s (3.7 MB)
✓ Uploaded to EAS 2s
✓ Computed project fingerprint

See logs: https://expo.dev/accounts/minchau_dev/projects/stemm-lab/builds/760b8c9b-55f2-4aa0-99e4-d69ed2e20db2

Waiting for build to complete. You can press Ctrl+C to exit.
✓ Build finished
```

Figure 8.1 — EAS build result and the installed app on an Android device.

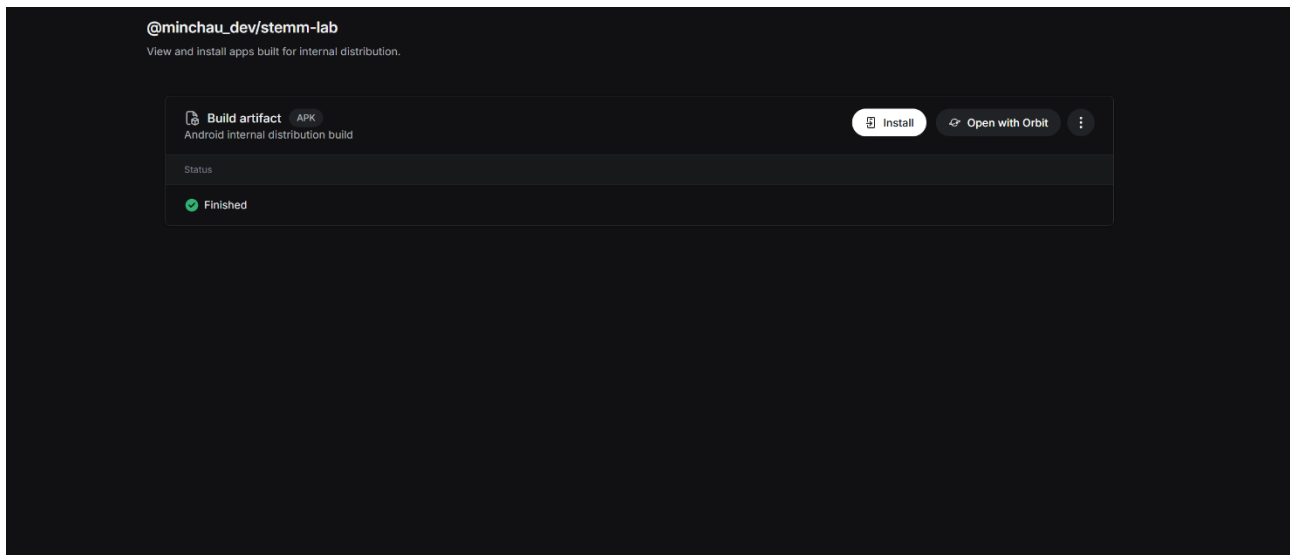


Figure 6.2 — APK build result

End of Testing and Deployment Report.